UCS3512 — MACHINE LEARNING LAB

All Topics: Question Patterns + Complete Programs

Exam preparation reference for the ML practical topics previously discussed.

Important source note: The exact original batch question papers are not attached in this conversation. Therefore, the questions below are realistic practice-style question wordings based on the topics previously discussed, not a claim that they reproduce your official papers word for word. Upload all batch question papers if you need exact question text and matching solutions.

Before running: Replace file paths and target-column names with the ones given in your question. Built-in datasets are used where possible so those programs do not require an external CSV.

Install: pip install numpy pandas matplotlib scipy scikit-learn

Standard answer structure: import libraries → load dataset → inspect/prepare → split → train → predict → evaluate/plot.

TOPIC 1 — Python packages: NumPy, SciPy, Scikit-learn, Matplotlib

Program 1

Example question wording: Write a Python program to demonstrate NumPy array operations, a SciPy statistical function, a Scikit-learn model and Matplotlib visualization.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.linear_model import LinearRegression

x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])
print("Array:", x)
print("Mean:", np.mean(x))
print("Sum:", np.sum(x))
print("Mode:", stats.mode(x, keepdims=True).mode[0])

model = LinearRegression()
model.fit(x.reshape(-1, 1), y)
print("Prediction for x=6:", model.predict([[6]])[0])

plt.plot(x, y, marker="o")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear relationship")
plt.grid(True)
plt.show()
```

Note: If SciPy is not installed, install it with pip install scipy.

TOPIC 2 — Loan amount prediction using Linear Regression

Program 2A — CSV dataset

Example question wording: Given a loan dataset, predict LoanAmount using Linear Regression and evaluate the model using MAE, MSE and R-squared.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = pd.read_csv("loan_data.csv")
print(df.head())
print(df.columns.tolist())

target = "LoanAmount"  # use the exact numeric target name
df = df.select_dtypes(include="number").dropna()
X = df.drop(columns=[target])
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, pred))
print("MSE:", mean_squared_error(y_test, pred))
print("R2 score:", r2_score(y_test, pred))

plt.scatter(y_test, pred)
plt.xlabel("Actual loan amount")
plt.ylabel("Predicted loan amount")
plt.title("Actual vs Predicted")
plt.show()
```

Note: If the dataset has categorical features, encode them before training. Ensure LoanAmount exists and is numeric.

TOPIC 3 — Handwritten character/digit recognition using a neural network

Program 3

Example question wording: Implement an MLP neural network to classify handwritten digits. Display a sample image and report accuracy and the confusion matrix.

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = load_digits()
X, y = data.data, data.target
X = X / 16.0

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = MLPClassifier(hidden_layer_sizes=(64,), max_iter=500,
                      random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, zero_division=0))

plt.imshow(data.images[0], cmap="gray")
```

```
plt.title("Sample digit: " + str(data.target[0]))
plt.axis("off")
plt.show()
```

Note: Uses scikit-learn's built-in 8x8 digit images; no separate image download is needed.

TOPIC 4 — Support Vector Machine (SVM): digits or spam classification

Program 4A — SVM on handwritten digits

Example question wording: Apply SVM with an RBF kernel to classify handwritten digits and evaluate the result.

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = load_digits()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = make_pipeline(StandardScaler(), SVC(kernel="rbf"))
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, zero_division=0))
```

Note: For email spam, use a CSV containing text and label columns; transform text with TfidfVectorizer and train SVC.

Program 4B — Email spam classification

Example question wording: Given a CSV with message and label columns, classify each email as spam or ham using TF-IDF and SVM.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

df = pd.read_csv("spam.csv")
text_col = "message"  # change to actual text column
label_col = "label"   # change to actual spam/ham label column
df = df.dropna(subset=[text_col, label_col])
X = df[text_col].astype(str)
y = df[label_col].astype(str)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = make_pipeline(TfidfVectorizer(), LinearSVC())
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, zero_division=0))
```

Note: If your CSV column names differ, change text_col and label_col.

TOPIC 5 — Diabetes prediction using Decision Tree

Program 5

Example question wording: Given a diabetes dataset, train a Decision Tree classifier to predict the diabetes outcome and print accuracy and confusion matrix.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt

df = pd.read_csv("diabetes.csv")
target = "Outcome"  # change to exact target column
df = df.dropna(subset=[target])
X = pd.get_dummies(df.drop(columns=[target]), drop_first=True)
X = X.fillna(X.median()).fillna(0)
y = df[target]
if y.dtype == "object":
    y = LabelEncoder().fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = DecisionTreeClassifier(random_state=42, max_depth=5)
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, zero_division=0))
plot_tree(model, feature_names=X.columns, filled=True, max_depth=2)
plt.show()
```

Note: Some diabetes datasets use class instead of Outcome. Check print(df.columns.tolist()).

TOPIC 6 — Random Forest, AdaBoost and Bagging

Program 6A — Compare Random Forest and AdaBoost

Example question wording: Train Random Forest and AdaBoost classifiers on the same dataset and compare their accuracy.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.metrics import accuracy_score, f1_score

df = pd.read_csv("data.csv")
target = "Outcome"  # replace with the target column
df = df.dropna(subset=[target])
X = pd.get_dummies(df.drop(columns=[target]), drop_first=True)
X = X.fillna(X.median()).fillna(0)
y = df[target]
if y.dtype == "object":
    y = LabelEncoder().fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
models = {
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "AdaBoost": AdaBoostClassifier(n_estimators=50, random_state=42)
}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(name, "Accuracy:", accuracy_score(y_test, pred))
    print(name, "F1:", f1_score(y_test, pred, average="weighted", zero_division=0))
```

Note: Replace data.csv and Outcome with your actual file and target.

Program 6B — Bagging classifier

Example question wording: Apply Bagging with Decision Trees and evaluate the classifier.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

df = pd.read_csv("data.csv")
target = "Outcome"
df = df.dropna(subset=[target])
X = pd.get_dummies(df.drop(columns=[target]), drop_first=True)
X = X.fillna(X.median()).fillna(0)
y = df[target]
if y.dtype == "object":
    y = LabelEncoder().fit_transform(y)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = BaggingClassifier(estimator=DecisionTreeClassifier(),
                          n_estimators=50, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
```

Note: If your installed scikit-learn is older and estimator is rejected, use base_estimator instead.

TOPIC 7 — K-Means clustering using Euclidean distance

Program 7

Example question wording: Apply K-Means to group numeric observations into three clusters. Scale the features and plot the first two dimensions.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

df = pd.read_csv("data.csv")
X = df.select_dtypes(include="number").dropna()
scaled = StandardScaler().fit_transform(X)

model = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = model.fit_predict(scaled)
print("Cluster labels:", labels)
print("Cluster centers:\n", model.cluster_centers_)
if len(set(labels)) > 1:
    print("Silhouette score:", silhouette_score(scaled, labels))

if scaled.shape[1] >= 2:
    plt.scatter(scaled[:, 0], scaled[:, 1], c=labels)
    plt.xlabel("Feature 1")
    plt.ylabel("Feature 2")
    plt.title("K-Means Clustering")
    plt.show()
```

Note: K-Means is unsupervised. Use numeric features only and change n_clusters if required.

TOPIC 8 — K-Nearest Neighbors (KNN)

Program 8

Example question wording: Implement KNN classification for a labelled dataset, scale features and evaluate accuracy, precision, recall, F1 and confusion matrix.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix

df = pd.read_csv("data.csv")
target = "Outcome"  # replace with exact target column
df = df.dropna(subset=[target])
X = pd.get_dummies(df.drop(columns=[target]), drop_first=True)
X = X.fillna(X.median()).fillna(0)
y = df[target]
if y.dtype == "object":
    y = LabelEncoder().fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
print("Precision:", precision_score(y_test, pred, average="weighted", zero_division=0))
print("Recall:", recall_score(y_test, pred, average="weighted", zero_division=0))
print("F1:", f1_score(y_test, pred, average="weighted", zero_division=0))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
```

Note: KNN depends on distance, so feature scaling is important.

TOPIC 9 — Q-Learning for grid path finding

Program 9

Example question wording: Implement Q-learning in a grid world with obstacles. Train the agent to reach the goal and print the learned path.

```
import numpy as np
import random

# 0 = open cell; 1 = obstacle
grid = np.array([[0, 0, 0, 0],
                 [0, 1, 0, 0],
                 [0, 1, 0, 0],
                 [0, 0, 0, 0]])
rows, cols = grid.shape
start, goal = (0, 0), (3, 3)
actions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
Q = np.zeros((rows * cols, len(actions)))

def state(pos):
    return pos[0] * cols + pos[1]

def move(pos, action):
    r, c = pos[0] + action[0], pos[1] + action[1]
    if r < 0 or r >= rows or c < 0 or c >= cols or grid[r, c] == 1:
        return pos
    return (r, c)

alpha, gamma, epsilon = 0.1, 0.9, 0.2
for episode in range(2000):
    pos = start
    for step in range(60):
        s = state(pos)
        if random.random() < epsilon:
            a = random.randrange(len(actions))
        else:
            a = int(np.argmax(Q[s]))
        nxt = move(pos, actions[a])
        reward = 10 if nxt == goal else (-5 if nxt == pos else -1)
        ns = state(nxt)
        Q[s, a] += alpha * (reward + gamma * np.max(Q[ns]) - Q[s, a])
        pos = nxt
        if pos == goal:
            break

pos, path, seen = start, [start], {start}
for step in range(30):
    a = int(np.argmax(Q[state(pos)]))
    nxt = move(pos, actions[a])
    if nxt == pos or nxt in seen:
        break
    path.append(nxt)
    seen.add(nxt)
    pos = nxt
    if pos == goal:
        break
print("Learned path:", path)
print("Reached goal:", path[-1] == goal)
```

Note: Q-learning update: Q(s,a) = Q(s,a) + alpha * (reward + gamma * max(Q(next_state)) - Q(s,a)).

TOPIC 10 — Dimensionality reduction using PCA

Program 10

Example question wording: Apply PCA to numeric dataset features, reduce them to two principal components, and display explained variance and a scatter plot.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

df = pd.read_csv("data.csv")
X = df.select_dtypes(include="number").dropna()
scaled = StandardScaler().fit_transform(X)
pca = PCA(n_components=2)
reduced = pca.fit_transform(scaled)

print("Original dimensions:", X.shape)
print("Reduced dimensions:", reduced.shape)
print("Explained variance ratio:", pca.explained_variance_ratio_)
print("Total variance retained:", pca.explained_variance_ratio_.sum())

plt.scatter(reduced[:, 0], reduced[:, 1])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA two-dimensional projection")
plt.show()
```

Note: If the question includes a target label, exclude that target from X before PCA.

TOPIC 11 — Mini-project: compare two algorithms

Program 11A — Compare KNN and SVM

Example question wording: Choose a labelled dataset, train KNN and SVM using the same split, and compare accuracy, precision, recall and F1-score.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

df = pd.read_csv("data.csv")
target = "Outcome"  # replace with actual target
df = df.dropna(subset=[target])
X = pd.get_dummies(df.drop(columns=[target]), drop_first=True)
X = X.fillna(X.median()).fillna(0)
y = df[target]
if y.dtype == "object":
    y = LabelEncoder().fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
models = {
    "KNN": make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5)),
    "SVM": make_pipeline(StandardScaler(), SVC(kernel="rbf"))
}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print("\n", name)
    print("Accuracy:", accuracy_score(y_test, pred))
    print("Precision:", precision_score(y_test, pred, average="weighted", zero_division=0))
    print("Recall:", recall_score(y_test, pred, average="weighted", zero_division=0))
    print("F1-score:", f1_score(y_test, pred, average="weighted", zero_division=0))
```

Note: For image datasets use pixel/features as X; for text use TfidfVectorizer; for IoT use sensor columns.

TOPIC 12 — Exploratory data analysis (EDA) and preprocessing

Program 12

Example question wording: Load a dataset, inspect its structure, check missing values and duplicates, plot distributions and correlations.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("data.csv")
print("First rows:\n", df.head())
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())
print("Info:")
df.info()
print("Summary:\n", df.describe(include="all"))
print("Missing values:\n", df.isnull().sum())
print("Duplicate rows:", df.duplicated().sum())

df = df.drop_duplicates()
numeric = df.select_dtypes(include="number")
numeric.hist(figsize=(10, 8))
plt.tight_layout()
plt.show()

if numeric.shape[1] >= 2:
    print("Correlation:\n", numeric.corr())
    plt.imshow(numeric.corr(), aspect="auto")
    plt.colorbar()
    plt.xticks(range(len(numeric.columns)), numeric.columns, rotation=90)
    plt.yticks(range(len(numeric.columns)), numeric.columns)
    plt.title("Numeric feature correlation")
    plt.tight_layout()
    plt.show()
```

Note: EDA is often a supporting step even when it is not the main algorithm question.

TOPIC 13 — Feature selection and cross-validation

Program 13

Example question wording: Use SelectKBest to select important features and cross-validation to estimate classifier performance.

```
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

df = pd.read_csv("data.csv")
target = "Outcome"  # replace with actual target
df = df.dropna(subset=[target])
X = pd.get_dummies(df.drop(columns=[target]), drop_first=True)
X = X.fillna(X.median()).fillna(0)
y = df[target]
if y.dtype == "object":
    y = LabelEncoder().fit_transform(y)

selector = SelectKBest(score_func=f_classif, k=min(5, X.shape[1]))
X_selected = selector.fit_transform(X, y)
print("Selected features:", X.columns[selector.get_support()].tolist())

X_train, X_test, y_train, y_test = train_test_split(
    X_selected, y, test_size=0.2, random_state=42, stratify=y
)
model = RandomForestClassifier(n_estimators=100, random_state=42)
scores = cross_val_score(model, X_selected, y, cv=5, scoring="accuracy")
print("CV scores:", scores)
print("Mean CV accuracy:", scores.mean())
model.fit(X_train, y_train)
print("Test accuracy:", accuracy_score(y_test, model.predict(X_test)))
```

Note: For a rigorous research pipeline, feature selection should be fitted inside each cross-validation fold to avoid leakage.

Common exam question formats

• **Implement:** Write a Python program to implement [algorithm] on [dataset].

• **Preprocess:** Load the dataset, inspect missing values, encode categorical columns and split the data.

• **Train and predict:** Train the model using fit() and predict test labels/values.

• **Evaluate:** Print suitable metrics and confusion matrix or regression errors.

• **Visualize:** Plot the data, clusters, PCA components, tree or actual-versus-predicted values.

• **Compare:** Train two algorithms on the same split and compare evaluation scores.

• **Explain:** State the purpose of the algorithm, target variable, features and meaning of the metrics.

Quick troubleshooting

• Check the CSV path and run print(df.columns.tolist()) to find the exact target name.

• If a dataset uses semicolons, load it with pd.read_csv('file.csv', sep=';').

• Use classifiers for class labels and regressors for continuous numeric values.

• Scale features for KNN, SVM, Logistic Regression, MLP and PCA.

• If train_test_split with stratify fails, inspect the number of rows in each class.

• This PDF gives practice-style question wording based on known topics. For exact batch-by-batch matching, the original batch question papers are required.

UCS3512 — MACHINE LEARNING LAB

Corrected target-aware codebook: robust target detection for supervised ML programs, with unsupervised topics kept target-free.

Important: The original batch question papers are not attached here. This guide corrects target handling and provides reusable code patterns; dataset column names and the prediction task must still be verified against your actual CSV.

1. Robust target-column detection — use in every supervised CSV program

This helper normalizes spaces, capitalization, underscores, and hyphens. It checks common target names, supports dataset-specific aliases, and raises a clear error if no target is found. Do not automatically select the last column unless you have verified that it is the label.

```
import pandas as pd

TARGET_CANDIDATES = [
    "target", "HeartDisease", "heart disease", "G3",
    "Type", "Class", "Outcome", "class", "axis",
    "label", "y", "diagnosis", "diabetes",
    "diabetes mellitus", "loan_status", "loan amount",
    "Loan_Status", "attrition", "Attrition",
    "quality", "species", "Survived", "charges"
]

def normalize_name(name):
    """Normalize column names for matching."""
    return "".join(ch.lower() for ch in str(name).strip()
                   if ch.isalnum())

def find_target_column(df, extra_candidates=None, target=None):
    """
    target: explicitly specify a column when you know the correct label.
    extra_candidates: dataset-specific aliases, checked before defaults.
    """
    if df.empty:
        raise ValueError("The dataset has no rows.")
    if target is not None:
        wanted = normalize_name(target)
        matches = [c for c in df.columns if normalize_name(c) == wanted]
        if not matches:
            raise ValueError(
                f"Target '{target}' not found. Available columns: {list(df.columns)}"
            )
        return matches[0]

    candidates = (extra_candidates or []) + TARGET_CANDIDATES
    column_map = {}
    for col in df.columns:
        key = normalize_name(col)
        # First exact normalized match wins
        column_map.setdefault(key, col)

    for name in candidates:
        key = normalize_name(name)
        if key in column_map:
            return column_map[key]

    raise ValueError(
        "Could not safely identify the target column. "
        f"Columns found: {list(df.columns)}. "
        "Set target='exact_column_name' after checking df.columns."
    )

# Example:
# df = pd.read_csv("your_dataset.csv")
# print("Columns:", df.columns.tolist())
# target = find_target_column(df, extra_candidates=["Outcome"])
# print("Selected target:", target)
```

Best practice: print `df.columns.tolist()` and confirm the selected target before training. For an ambiguous dataset, set `target='the_exact_column_name'` manually.

2. Standard supervised CSV workflow — use for classification

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler, LabelEncoder
```

```
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.tree import DecisionTreeClassifier

df = pd.read_csv("your_dataset.csv")
df.columns = df.columns.astype(str).str.strip()
print("Shape:", df.shape)
print("Columns:", df.columns.tolist())
print(df.head())

target = find_target_column(df, extra_candidates=["Outcome"])  # or target="Class"
df = df.dropna(subset=[target]).copy()
X = df.drop(columns=[target])
y = df[target]

if X.shape[1] == 0:
    raise ValueError("No feature columns remain after removing the target.")
if y.nunique() < 2:
    raise ValueError("Classification target has fewer than 2 classes.")

# Remove rows/columns that are completely empty
X = X.dropna(axis=1, how="all")
numeric_cols = X.select_dtypes(include="number").columns.tolist()
categorical_cols = X.select_dtypes(exclude="number").columns.tolist()

numeric_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
categorical_pipe = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])
preprocess = ColumnTransformer([
    ("num", numeric_pipe, numeric_cols),
    ("cat", categorical_pipe, categorical_cols)
])

# Stratify only when every class has enough samples
stratify_y = y if y.value_counts().min() >= 2 else None
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=stratify_y
)
model = Pipeline([
    ("preprocess", preprocess),
    ("model", DecisionTreeClassifier(random_state=42))
])
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Target column:", target)
print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, zero_division=0))
```

For SVM, KNN, Logistic Regression, Random Forest, AdaBoost, and MLP, keep the same data-loading, target detection, preprocessing, split, and metrics. Replace only the estimator in the final pipeline. Some estimators may need their own parameters.

3. Standard supervised CSV workflow — use for regression

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.linear_model import LinearRegression
import numpy as np

df = pd.read_csv("your_dataset.csv")
df.columns = df.columns.astype(str).str.strip()
print("Columns:", df.columns.tolist())

# Example: G3 for student grades, or specify the exact numeric target.
target = find_target_column(df, extra_candidates=["G3"], target=None)
df = df.dropna(subset=[target]).copy()
y = pd.to_numeric(df[target], errors="coerce")
valid = y.notna()
df, y = df.loc[valid].copy(), y.loc[valid]
X = df.drop(columns=[target])
X = X.dropna(axis=1, how="all")

numeric_cols = X.select_dtypes(include="number").columns.tolist()
categorical_cols = X.select_dtypes(exclude="number").columns.tolist()
preprocess = ColumnTransformer([
    ("num", Pipeline([
```

```
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]), numeric_cols),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore"))
    ]), categorical_cols)
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = Pipeline([
    ("preprocess", preprocess),
    ("model", LinearRegression())
])
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Target column:", target)
print("MAE:", mean_absolute_error(y_test, pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, pred)))
print("R2:", r2_score(y_test, pred))
```

Do not use the classification workflow for a numeric grade/loan-amount target. Conversely, do not use regression metrics for a categorical outcome.

4. Dataset-specific target guidance

Heart / heart disease datasets

Candidate labels include `target`, `HeartDisease`, `Heart Disease`, `output`, or `condition`. These are not interchangeable across all files. Inspect the columns and choose the column that actually represents the outcome. If needed, use `target = find_target_column(df, target='HeartDisease')`.

Student performance (`student-mat.csv`)

For predicting final grade, use `G3` as the target. If predicting a different grade, specify that column explicitly. `G3` is numeric, so Linear Regression is appropriate for regression tasks; Decision Tree can be either regressor or classifier depending on how the question defines the output.

Diabetes datasets

Possible labels include `Outcome`, `class`, or `diabetes`. Verify which column means diabetic/non-diabetic. Some datasets encode the label as Yes/No or positive/negative; the classification pipeline can handle categorical labels.

Glass identification

The class label is often named `Type`, `Class`, or `Type of glass`. Confirm the exact label column; all remaining measurement columns are features. PCA can be applied to features after preprocessing, but the class label should not be included in PCA input.

HR attrition

Common label: `Attrition`. If the CSV has `Attrition` as Yes/No, use classification. Remove employee IDs and other non-predictive identifiers from features when appropriate.

Loan prediction

A loan amount is a regression target only if the question asks to predict a numeric amount. Approval/status is classification. Do not use `LoanAmount` as the target for a loan-approval question.

5. Replace only the estimator for each algorithm

```
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, BaggingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neural_network import MLPClassifier

# In the classification pipeline, replace DecisionTreeClassifier(...) with one:
SVC(kernel="rbf", C=1.0)
KNeighborsClassifier(n_neighbors=5)
DecisionTreeClassifier(random_state=42)
RandomForestClassifier(n_estimators=100, random_state=42)
AdaBoostClassifier(random_state=42)
BaggingClassifier(n_estimators=50, random_state=42)
LogisticRegression(max_iter=2000)
```

```
MLPClassifier(hidden_layer_sizes=(100,), max_iter=1000, random_state=42)

# For regression:
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, AdaBoostRegressor
DecisionTreeRegressor(random_state=42)
RandomForestRegressor(n_estimators=100, random_state=42)
AdaBoostRegressor(random_state=42)
```

Important: in scikit-learn, some AdaBoost/Bagging constructor parameters differ by library version. The simple constructors above are more portable; check your installed version if your lab machine uses an older scikit-learn.

6. PCA, K-Means, and Q-learning — do not force target detection

PCA

PCA is unsupervised dimensionality reduction. Do not include the target/label column in the PCA input. Select feature columns, impute missing values, encode categorical features, scale them, then fit PCA. If a label is available, keep it separately only for coloring/evaluating plots.

```
# X should contain features only; target/label must be excluded.
target = find_target_column(df)  # only if this dataset has a known label
X = df.drop(columns=[target])
# Then preprocess X and apply PCA to the transformed feature matrix.
from sklearn.decomposition import PCA
pca = PCA(n_components=2, random_state=42)
X_2d = pca.fit_transform(X_scaled)
print("Explained variance:", pca.explained_variance_ratio_)
```

K-Means

K-Means has no target column. Use only feature data, scale it, fit `KMeans`, and evaluate clusters using inertia/silhouette where appropriate. Do not call `find_target_column` unless you are separately comparing predicted clusters with a known label.

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import make_pipeline

X = df.select_dtypes(include="number").dropna(axis=1, how="all")
cluster_model = make_pipeline(
    SimpleImputer(strategy="median"),
    StandardScaler(),
    KMeans(n_clusters=3, random_state=42, n_init=10)
)
clusters = cluster_model.fit_predict(X)
print("Cluster assignments:", clusters)
```

Q-learning

Q-learning uses states, actions, rewards, and next states, not a dataset target column. Leave target detection out of the Q-learning grid-world program.

7. Important checks before the exam

• Print `df.shape`, `df.columns.tolist()`, and `df.head()` before choosing the target.

• Print the selected target: `print('Target:', target)`.

• Check `df[target].value_counts(dropna=False)` for classification or `df[target].describe()` for numeric regression.

• Drop missing target rows before splitting; never impute the target as though it were a feature.

• Create `X = df.drop(columns=[target])` and `y = df[target]`; this prevents target leakage.

• Fit preprocessing only on training data by putting it inside a Pipeline.

• Do not use target detection in built-in datasets when the label is already explicit, e.g. `load_digits().target`.

• For datasets with ambiguous names, explicitly set `target='exact column name'` after inspecting the CSV.

• If a target candidate is found but does not represent the requested outcome, override it manually.

• For PCA/K-Means/Q-learning, use their unsupervised/reinforcement-learning workflow rather than forcing a target.

This corrected guide focuses on preventing wrong-target selection and train/test leakage. It cannot guarantee every external CSV will run unchanged without knowing its actual column names, data types, and installed package versions.